

Build a Grievance & Land Records Website

A complete step-by-step guide for students, from zero to a live website, using free online tools and an AI coding assistant

Example project: MCL Land & Revenue Management & Grievance Redressal System (MCL-LRMS), built by an MCL team at an IIM Sambalpur MDP.

Live example: <https://mcl-grievance-redressal-team.vercel.app>



No programming knowledge is needed. The student describes the website in plain English. The AI assistant writes the code. The student reviews, approves and tests.

0. What the student will build, and which tools are used

The website (3 pages):

- **Home / Citizen portal** ([index.html](#)) → register a grievance, get a reference number, track its status.
- **Officer Dashboard** ([dashboard.html](#)) → secure sign-in, summary cards, charts, filters, assign, field verification, dispose, reports, audit trail.
- **Map & Field page** ([field.html](#)) → find a plot on a satellite map, AI reading of scanned maps, GPS field verification with photos.

Tool / website	What it does in this project	Web address	Cost
GitHub	Stores all the website files and their full history ("the project folder in the cloud").	github.com	Free
Supabase	The online database: saves grievances, field visits and officer logins.	supabase.com	Free plan
Vercel	Publishes the website on the internet with a public link. Updates it automatically.	vercel.com	Free (Hobby)
Claude Code	The AI assistant that writes the code, saves it to GitHub and opens pull requests.	claude.ai/code	Paid Claude plan
Web browser	Chrome or Edge, on a laptop and a mobile phone, for testing.	-	Free

Map services	OpenStreetMap and Esri satellite tiles. They are loaded automatically by the map library (Leaflet).	-	Free
---------------------	---	---	------

Key words in one line each:

- **Repository (repo)**: a project folder on GitHub.
- **Branch**: a separate working copy of the files, where changes are made safely.
- **Pull request (PR)**: a request to copy the changes from a branch into the main files.
- **Merge**: approving a pull request, so the changes become official.
- **main**: the official branch. Vercel publishes whatever is in *main*.
- **SQL**: the language used to create database tables. Students only copy and paste it.

1. Create the accounts (one time, about 20 minutes)

1.1 GitHub

- Open github.com → **Sign up** → enter email, password and a username → verify the email.

1.2 Supabase

- Open supabase.com → **Start your project** → **Continue with GitHub** → allow access.

1.3 Vercel

- Open vercel.com → **Sign Up** → choose **Hobby** (free) → **Continue with GitHub** → allow access.

1.4 Claude

- Open claude.ai → sign up → choose a plan that includes **Claude Code**.
- Open claude.ai/code → **Connect GitHub** → install the **Claude GitHub App** → allow it on your repository (or "All repositories").

Common problem: if Claude later says "push failed / error 403 / no access", open claude.ai/connect-github. Reconnect GitHub there, and make sure the Claude GitHub App is installed on that repository.

2. Create the GitHub repository and the starter files

- GitHub → top-right + → **New repository**.
- Repository name, e.g. [grievance-redressal-team](#) → choose **Public** or **Private** → tick **Add a README file** → **Create repository**.
- Add three starter files: **Add file** → **Create new file** → type the file name → paste the content → **Commit changes**.

2.1 File 1: `config.js` (database settings, filled in later)

```
// config.js - database settings. Only the Project URL and the PUBLISHABLE key go here.
// NEVER put a secret key, a service_role key or the database password in this file.
window.SUPABASE_URL = "PASTE_PROJECT_URL_HERE";
window.SUPABASE_PUBLISHABLE_KEY = "PASTE_PUBLISHABLE_KEY_HERE";
```

2.2 File 2: `index.html` (a simple placeholder page)

```
<!DOCTYPE html><html><head><meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<title>Coming soon</title></head>
<body><h1>Our team's tool is coming soon</h1></body></html>
```

2.3 File 3: `CLAUDE.md` (the rule book the AI reads in every session)

This is the most important file. It tells the AI who the students are, what to build and which rules to follow. A shortened version:

```
# CLAUDE.md - read this first, in every session
## About us
- We are students, NOT programmers. Explain everything in plain English, short sentences.
## What we are building
- A tool with at most 3 pages: index.html, dashboard.html and one more page.
- Every record has location, urgency (Low/Medium/High) and status (Open/In progress/Resolved).
- All data is MADE UP. Never add real names, phone numbers or ID numbers.
## Technical rules
1. Plain HTML, CSS and JavaScript only. No frameworks, no build step.
2. Vercel publishes the site from the main branch. Use relative links only.
3. Load Supabase from the jsDelivr CDN, then config.js.
4. The Project URL and publishable key live only in config.js. Never use a secret key.
5. For charts, load Chart.js from the jsDelivr CDN.
6. If anything fails, show a friendly message with the actual error text.
7. Every page must work well on a mobile phone.
## Database rules
- We run all SQL ourselves in the Supabase SQL Editor. Give SQL as ONE block,
  saved in the database folder: 01-setup.sql, 02-..., 03-...
- One table, Row Level Security on, never drop tables or delete rows.
## How to work with us
- One change at a time. After each change: 3 short bullets - what changed, what to test.
- Commit and push at every stopping point.
## Progress Log (newest entry at the bottom)
- Phase 0: starter files. Next: Phase 1.
```

Why a Progress Log? Each new AI session starts with no memory. It reads CLAUDE.md first, so the log is the "handover register" between sessions and between students.

3. Create the Supabase database project

- Supabase → **New project** → choose the organisation → Project name (e.g. *grievance-demo*).
- Database password → click **Generate a password** → **save it privately**. It is never put into the website.
- Region → **South Asia (Mumbai)** → **Create new project** → wait about 2 minutes.

3.1 Copy the two settings the website needs

→ **Project URL:** click the **Connect** button at the top, or go to Project Settings → **Data API**. It looks like `https://abcdefghijklmnop.supabase.co`.

→ **Publishable key:** Project Settings → **API Keys** → copy the key that starts with `sb_publishable_`.

Never copy the *secret* key, the *service_role* key or the database password into the website or the chat. The publishable key is safe to be public.

Shortcut: the Project URL can also be built from the browser address bar. In `supabase.com/dashboard/project/abcdefghijklmnop`, the code at the end gives `https://abcdefghijklmnop.supabase.co`.

4. Connect Vercel to publish the website

→ Vercel → **Add New...** → **Project** → find your GitHub repository → **Import**.

→ Framework Preset → **Other** → leave all other settings as they are → **Deploy**.

→ After about 1 minute you get a link like `https://grievance-redressal-team.vercel.app` → open it → the placeholder page appears.

From now on Vercel works automatically. Every time something is merged into *main* on GitHub, Vercel publishes the new version within 1-2 minutes. For each pull request, Vercel also posts a "Preview" link as a comment, so you can test before merging.

5. Build the website with Claude Code

- Open `claude.ai/code` → choose your repository → start a new session.
- First message: *"Read CLAUDE.md and summarise the Progress Log in 3 bullets."*
- Then paste the **project brief**: what problem, who uses it, which pages, which fields, which reports. Ask for a working **demo with clearly marked sample data**.

5.1 A good project brief contains

- Background → the real problem (e.g. acquired land is hard to locate; grievances are slow to dispose).
- Users → citizens (register and track) and officers (examine, verify, dispose).
- Grievance form fields → name, mobile, village, khata, plot, type, description, documents, photo, consent.
- Status flow → Submitted → Under Scrutiny → Field Verification Required → Under Examination → Action Taken → Disposed.
- Officer features → dashboard cards, filters, assignment, field verification, disposal report, audit trail.
- Map → village, khata and plot search, satellite view, GPS, photos.
- Limits → "AI must not decide ownership; final decision with officers"; "all data made up".

5.2 What Claude does next

- Writes the pages, the shared styles and the database SQL files → tests them in its own browser → saves the work on a **branch** → pushes it to GitHub.
- Ask: *"Create a pull request for me."* → Claude replies with a link like `github.com/<user>/<repo>/pull/1`.

Good habits when talking to the AI

- One change per message ("make the home page cleaner", "make photos optional").
- When something fails, copy the exact red error text from the page and paste it into the chat.
- Ask "what should I test now?" after each change.
- At the end of a working session, ask it to "add a save point to the Progress Log".

6. Merge the pull request (make the change official)

- Open the pull request link → read the description → optionally open the Vercel **Preview** link in the comments to test first.
- Click the green **Merge pull request** → **Confirm merge**.
- Wait 1-2 minutes → open your Vercel link → refresh (on a phone, pull down to reload).

If there is no pull request link: GitHub → **Pull requests** tab → **New pull request** → **base: main, compare: claude/...** (Claude's branch) → **Create pull request** → then merge.

7. Set up the database tables (Data Keeper's job)

- On GitHub, open `database/01-setup.sql` → click the **copy** icon (top-right of the file).
- Supabase → left menu **SQL Editor** → **New query** → paste → **Run** → it must say "**Success. No rows returned**".
- Repeat for every later file in order: `02-officer-login.sql`, then `03-...`.
- Check: left menu **Table Editor** → the new table appears (e.g. `lrms_records`). It is empty at first.

The SQL files in this project are written so that running them again is safe. They never delete data.

8. Fill in config.js (connect the website to the database)

- GitHub → open `config.js` → click the  **pencil** (Edit).
- Replace `PASTE_PROJECT_URL_HERE` with the Project URL and `PASTE_PUBLISHABLE_KEY_HERE` with the publishable key. **Keep the quote marks.**
- **Commit changes...** → "Commit directly to the main branch" → **Commit changes.**

The finished file must look exactly like this, with your own values:

```
window.SUPABASE_URL = "https://abcdefghijklmno.supabase.co";  
window.SUPABASE_PUBLISHABLE_KEY = "sb_publishable_XXXXXXXXXXXXXXXXXXXX";
```

Two mistakes we actually made (avoid them):

- The URL ended with `/rest/v1/`. Remove that; the URL must end with `.supabase.co`.
- The start of the key line (`window.SUPABASE_PUBLISHABLE_KEY = "`) was deleted by accident. That breaks the whole file.

9. Create officer logins (only if the project has an officer sign-in)

- Supabase → **Authentication** → **Users** → **Add user** → **Create new user.**
- Email (can be made up, e.g. `officer1@demo.in`) → a password of at least 8 characters → tick **Auto Confirm User** → **Create user.**
- Give each team member their own login. Share passwords privately, never in a public chat or file.
- To change a password later: Authentication → Users → click the user → set a new password.

The AI assistant **cannot** create these accounts and never sees the passwords. This is deliberate and keeps the system safe.

10. Test the live website (example test script)

#	Where	What to do → what should happen
1	Home → Register Grievance	Fill only the * fields (mobile: 10 digits starting 6-9, e.g. 90000000001) → Submit → a reference number appears, e.g. <code>MCL-KA-GRV-2026-000125</code> .
2	Supabase → Table Editor	The new grievance is visible as a row.
3	Officer Dashboard	Wrong password → refused. Right password → choose a role → dashboard opens; the new grievance is in the list.
4	Grievance window	Schedule field verification → assign to a field officer.
5	Find Plot & Field	Village → Khata → Plot → Locate Plot → the plot is highlighted on the map.
6	Field Verification (phone)	Start → capture GPS → mark 4 boundary points → take a photo → choose result → Submit → the plot colour changes.
7	Dashboard	Record action taken → Dispose (confirm "Are you sure?") → print the Disposal Report (Save as PDF).
8	Home → Track Grievance	Reference number + mobile → status shows Disposed , with public remarks only.
9	Phone + laptop	No sideways scrolling, buttons are large, the Hindi button works.

11. Making changes later (the routine cycle)

Ask Claude for ONE change → Claude tests & pushes → Pull request link →
 Check Preview → Merge → Wait 1-2 min → Test live

→ If the change includes a new SQL file (e.g. `03-... .sql`) → run it in the Supabase SQL Editor **right after merging**.

→ At the end of the day → ask Claude: "Add a save point to the Progress Log and push."

→ A new session (or another student) → first message: "Read CLAUDE.md and summarise the Progress Log."

12. Keeping it running "whenever required"

Situation	What to do
Supabase project paused (free projects pause after about a week without use)	Supabase → open the project → Restore project → wait a few minutes → the website works again. Data is kept. Open the site a day before any presentation.

Website shows an old version	Vercel → project → Deployments → the newest one must say Ready . If it says Error, open it and copy the message for Claude. Then refresh the browser (Ctrl + F5).
A change broke something	GitHub → Pull requests → Closed → open the last merged PR → Revert → merge the revert PR. Or tell Claude: "undo the last change".
Need the website link	Vercel → project → Domains , or the Visit button.
Forgot an officer password	Supabase → Authentication → Users → click the user → set a new password.
Before a presentation	Open the site on the projector laptop and a phone → sign in → register one test grievance → keep the demo script printed.
Clean demo data	Do not delete rows (rule). Add new clearly-labelled test records instead, or ask Claude for a "filter out test records" option.

13. Troubleshooting: error message → cause → fix

Message on the page	Cause	Fix
"Database settings are not filled in" / "database is not connected"	config.js still has the PASTE_... text, or has a typo	Section 8. Check the URL ends with .supabase.co and the key line is complete.
"Could not find the function public.register_grievance..."	The second SQL file was not run	Run <code>02-officer-login.sql</code> in the SQL Editor.
"relation ... does not exist"	The first SQL file was not run	Run <code>01-setup.sql</code> , then 02.
"permission denied for table ..." / "row-level security"	A grant or policy is missing, or the officer is not signed in	Re-run the SQL files in order. Sign in again.
"Invalid login credentials"	Wrong email/password, or the user was not created	Section 9. Tick Auto Confirm User.
"Please complete the fields marked *"	A required field is empty, or the mobile number is not 10 digits starting 6-9	Fill the fields shown in red.
Claude: "push failed / 403"	The Claude GitHub App is not connected to the repo	Open <code>claude.ai/connect-github</code> → reconnect → install the app on the repo.
Map or charts do not appear	No internet, or a blocked CDN on office Wi-Fi	Try mobile data. The libraries load from cdn.jsdelivr.net.

14. Safety and ethics rules for students

- → Use **made-up data only** in a demo. No real names, phone numbers, ID numbers or official figures.
- → Never share the **database password**, the **secret / service_role key**, or officer passwords.

- → The publishable key and the Project URL are designed to be public. They are fine in config.js.
- → The system **supports** decisions. It must never declare ownership, title or possession. Officers decide.
- → AI results (text read from maps, confidence values) must be shown as **suggestions for human review**, never as proof.
- → Clearly label demo maps and boundaries as **DEMO / not real parcels**.

15. One-page checklist

- ☐ GitHub, Supabase, Vercel and Claude accounts created; Claude GitHub App installed on the repo
- ☐ Repository created with README, config.js, index.html and CLAUDE.md
- ☐ Supabase project created (Mumbai region); database password saved privately
- ☐ Vercel project imported from GitHub → placeholder page visible at the .vercel.app link
- ☐ Claude Code session started → project brief given → pull request created
- ☐ Pull request merged → new pages visible on the live link
- ☐ 01-setup.sql and 02-...sql run in the SQL Editor → "Success"
- ☐ config.js filled in (URL ends with .supabase.co; key starts with sb_publishable_)
- ☐ Officer users created (Auto Confirm ticked)
- ☐ Test script (Section 10) passed on a laptop and a phone
- ☐ Save point written in the CLAUDE.md Progress Log

Prepared from the MCL-LRMS project (IIM Sambalpur MDP, October 2026). Screens and menu names of GitHub, Supabase, Vercel and Claude may change slightly over time; the steps stay the same.